ACTIVITAT 1: Cerques

Autors: Pablo Martín i Lidia Gil
Grup: LaGranPropuesta
Assignatura: Projecte de Programació

Documentació de la heurística

La heurística avançada implementada realitza una estimació optimista del cost necessari per a que els agents arribin a la sortida, considerant la col·laboració entre ells per recollir les claus pendents. El càlcul es realitza sense tenir en compte els obstacles de l'escenari.

Primer de tot, es mira si queden claus per recollir, si és així, l'heurística asigna de manera cada clau pendent a l'agent més proper mitjançant el càlcul de la distancia de Manhattan. En cada iteració, selecciona la parella agent-clau amb menor distància, acumula aquesta distància al total i actualitza la posició de l'agent a la ubicació de la clau, simulant que s'ha desplaçat fins allà. Aquest procés es repeteix fins que totes les claus han estat assignades als agents.

Segonament, si totes les claus han estat recollides, es calcula la distància mínima de qualsevol agent fins la posició de la sortida, també mitjançant la distància de Manhattan. Aquest valor s'afegeix al total acumulat a la fase de recollida de claus.

El resultat final és una estimació admissible que considera la col·laboració òptima entre agents, tot i que no té en compte la presència d'obstacles com els murs o les portes tancades durant els càlculs de distància.

[1] Calculeu el factor d'embrancament mig (real) del mapa C per BFS usant LNT i sense fer-la servir. Justifiqueu les avantatges d'usar la LNT respecte la l'opció que només evitar cicles en el camí actual.

Si afegim a la nostra classe CercaBFS la següent configuració per detectar el factor d'embrancament real:

```Java
package edu.epsevg.prop.ac1.cerca;

import java.util.ArrayDeque;
import java.util.HashMap;
import java.util.Map;

import edu.epsevg.prop.ac1.model.Mapa;
import edu.epsevg.prop.ac1.model.Moviment;
import edu.epsevg.prop.ac1.resultat.ResultatCerca;

public class CercaBFS extends CercaDopada {
    private int nodesGenerats = 0;
    private int nodesExpandits = 0;
```

```java
    public CercaBFS(boolean usarLNT) { super(usarLNT); }

    @Override
    public void ferCerca(Mapa inicial, ResultatCerca rc) {
        ArrayDeque<Node> LNO = new ArrayDeque<>();
        Map<Mapa, Node> LNT = new HashMap<>();
        Node na = new Node(inicial, null, null, 0, 0);
        LNO.push(na);
        nodesGenerats = 1; // Node inicial
        nodesExpandits = 0;
        boolean hiHaSolucio = false;

        while(!LNO.isEmpty()){
            na = LNO.removeFirst();
            rc.incNodesExplorats();

            if(na.estat.esMeta()){
                hiHaSolucio = true;
                break;
            }
            else{
                boolean expandir = tractarLNTDepth(na, LNT, rc);

                if (expandir) {
                    nodesExpandits++;
                    int fillsGenerats = 0;
                    for (Moviment m : na.estat.getAccionsPossibles()){
                        Node fill = new Node(na.estat.mou(m), na, m, na.depth +
1, 0);

                        LNO.addLast(fill);
                        fillsGenerats++;
                    }
                    nodesGenerats += fillsGenerats;
                }
            }
            rc.updateMemoria(LNO.size() + LNT.size());
        }

        // Resultat final
        if (nodesExpandits > 0) {
            double factorFinal = (double)(nodesGenerats - 1) / nodesExpandits;
            System.out.printf("FACTOR EMBRANCAMENT FINAL: %.3f%n",
factorFinal);
            System.out.printf("Nodes generats: %d, Nodes expandits: %d%n",
```

```
                                nodesGenerats, nodesExpandits);
            }

            if(hiHaSolucio){
                System.out.println("Profunditat: " + na.depth);
                rc.setCami(reconstruirCami(na));
            }
        }
    }
```

En cas d'ús de la LNT, la solució ens proporciona el següent output: **FACTOR EMBRANCAMENT FINAL: 5.001**

En el cas de no usar la LNT, no podem saber el factor d'embrancament degut a l'output: **OUT OF MEMORY ERROR**

L'ús de la Llista de Nodes Tancats (LNT) evita que un mateix node es generi diverses vegades per camins diferents, reduint així la redundància de l'arbre d'exploració. En canvi, si només s'eviten els cicles en el camí actual, un mateix node pot ser expandit múltiples vegades, provocant un augment considerable de memòria i un arbre molt més gran. La LNT garanteix que cada node s'expandeixi només una vegada, millorant l'eficiència de la cerca.

[2] [1.5p] Analitzeu comparativament els resultats dels diferents algorismes (només amb la versió LNT), justificant-ne el comportament. Tingueu en compte els següents aspectes: Si acaben la cerca o no (i si no acaben, indicar el motiu: temps excessiu, manca de memòria...) Si han trobat la solució, i en aquest cas, si és l'òptima. El nombre de nodes explorats. (en el cas de IDS, cal acumular per cada iteració que no troba solució) La memòria pic utilitzada (s'aproximarà com la mida màxima LNO (+ la mida de la LNT si s'utilitza)) Nodes tallats per evitar cicles (prunning) Temps d'execució. Podeu usar quadres de resum i gràfiques.

Després de fer totes les cerques amb tots els mapes proporcionats en l'enunciat, a continuació es mostren les estadístiques resultants:

Taula comparativa general

Algoritme	Mapa	Trobat	Longitud	Nodes Explorats	Nodes Tallats	Memoria Pic	Temps Ms
BFS	mapA	YES	5	483	329	468	12
DFS	mapA	YES	22	34	11	90	0
IDS	mapA	YES	5	874	514	143	3
A* Basica	mapA	YES	5	82	38	199	9
A* Avancada	mapA	YES	5	16	3	63	1
BFS	mapB	YES	31	454	271	190	1
DFS	mapB	YES	41	166	77	130	1
IDS	mapB	YES	31	15828	9248	214	26
A* Basica	mapB	YES	31	361	210	171	3
A* Avancada	mapB	YES	31	245	125	190	3
BFS	mapC	YES	37	62762	49649	15930	84
DFS	mapC	YES	50	375987	294320	13091	337
IDS	mapC	YES	37	3645843	2832596	13066	2953
A* Basica	mapC	YES	37	54783	43343	14325	132
A* Avancada	mapC	YES	37	31368	24432	10477	96
BFS	mapD	YES	79	808820	695002	122294	1731
DFS	mapD	NO)	-	5096961	4350913	82506	8945
IDS	mapD	NO)	-	61817334	52526719	82506	106158
A* Basica	mapD	YES	79	761181	654192	115328	3177
A* Avancada	mapD	YES	79	117659	96458	53426	559

1. CERCA BFS

Algoritme	Mapa	Trobat	Longitud	Nodes Explorats	Nodes Tallats	Memoria Pic	Temps Ms
BFS	mapA	YES	5	483	329	468	12
BFS	mapB	YES	31	454	271	190	1
BFS	mapC	YES	37	62762	49649	15930	84
BFS	mapD	YES	79	808820	695002	122294	1731

Ha trobat sempre la solució i la més òptima en els 4 mapes, però respecte a altres algoritmes és del que més nodes explora per sota del algoritme IDS, això és degut al seu funcionament d'expansió per nivells. La memòria pic utilitzada, per tant, al ser la suma de la màxima LNO i la LNT és la més alta de tots els mapes menys en el mapaB i respecte als nodes tallats, sempre és el segon en quantitat més alta per sota del IDS gràcies a la utilització de la LNT.

2. CERCA DFS

Mapa	Trobat	Longitud	Nodes Explorats	Nodes Tallats	Memòria Pic	Temps (ms)
mapA	YES	22	34	11	90	0
mapB	YES	41	166	77	130	1
mapC	YES	50	375987	294320	13091	337
mapD	NO	-	5096961	4350913	82506	8945

En la segona cerca, observem que DFS (Depth-First Search) és un algorisme que no garanteix trobar la solució més òptima, sinó una solució qualsevol dins dels límits de profunditat establerts. Aquesta afirmació la podem trobar a la taula comparativa a l'inici d'aquest apartat.

En els casos de mapA, mapB i mapC, la cerca finalitza correctament i troba una solució.
En canvi, en el cas de mapD, obtenim el següent resultat:

```shell
>SOLUCIO:null
       DFS -> Resultat: sense solució, explorats=5096961, tallats=4350913,
memòria=82506, temps=14359ms
```

En aquest cas, el resultat és **NO** perquè la cerca s'ha limitat a una profunditat màxima de 50, mentre que el camí òptim té una longitud de 79, és a dir, que com a mínim la primera solució es trobarà a profunditat 79. Per tant, l'algorisme DFS s'atura abans d'arribar a la solució i retorna "no trobat".

L'augment de nodes explorats és molt notable a mesura que explora nodes més profunds del mapa. En mapD, encara que no arriba a trobar solució, podem observar que s'exploren més de 5 milions de nodes, amb més de 4 milions de talls (pruning) per evitar cicles o tornar a estats ja visitats. A més, requereix poca memòria en comparació amb altres algorismes, ja que només desa el camí actual (LNO), ja que té un cost en memòria de b·d. Finalment, el temps augmenta dràsticament amb la mida del mapa i la profunditat.

3.CERCA IDS

Mapa	Trobat	Longitud	Nodes Explorats	Nodes Tallats	Memòria Pic	Temps (ms)
mapA	YES	5	874	514	143	3
mapB	YES	31	15828	9248	214	26
mapC	YES	37	3645843	2832596	13066	2953
mapD	NO	-	61817334	52526719	82506	106158

La cerca combina els avantatges del BFS i el DFS realitzant cerques en profunditat limitada iterativament amb un límit que incrementa progressivament. Tot i que la cerca troba el resultat òptim en els mapes petits no troba solució als grans i també explora una quantitat de nodes desmesurada degut a la reiterada expansió dels mateixos estats en cada increment de profunditat que fa. Aquest comportament es reflecteix en el temps d'execució elevat i un gran nombre de nodes tallats, tot i que manté un ús de memòria moderat, gràcies a que fa cerca basada en profunditat com un DFS. En mapes complexos segons els resultats obtinguts no és un algoritme viable i eficient.

4.CERCA A* BÀSICA

Mapa	Trobat	Longitud	Nodes Explorats	Nodes Tallats	Memòria Pic	Temps (ms)
mapA	YES	5	82	38	199	9
mapB	YES	31	361	210	171	3
mapC	YES	37	54783	43343	14325	132
mapD	YES	79	761181	654192	115328	3177

La heurística utilitzada ha sigut la distancia de Manhattan a la clau més propera als agents sempre que hi havia claus pendents per recollir i sinó la mínima distancia de Manhattan dels agents a la sortida. Tot i que la heurística no té en compte els objectes, ha trobat sempre la solució i la més òptima en els 4 mapes, explora bastant menys que el BFS però pateix una petita penalització de temps en mapes petits pel càlcul de la heurística.

5.CERCA A* AVANÇADA

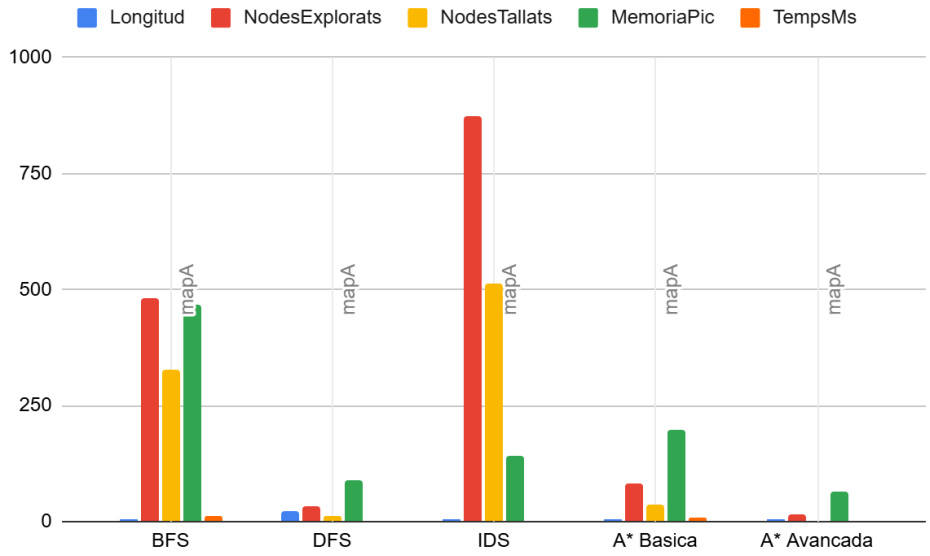
Mapa	Trobat	Longitud	Nodes Explorats	Nodes Tallats	Memòria Pic	Temps (ms)
mapA	YES	5	16	3	63	1
mapB	YES	31	245	125	190	3
mapC	YES	37	31368	24432	10477	96
mapD	YES	79	117659	96458	53426	559

Al haver fet una millora de la heurística, hi ha hagut un descens respecte al número de nodes explorats, en el pic de memòria i en el temps en comparació amb la cerca bàsica, especialment en mapes grans. Ha obtingut la millor eficiència general i aconseguint sempre la solució òptima. A més, degut a la poca exploració de nodes a aconseguit bons temps comparats amb els altres algoritmes implementats.

ANNEXOS

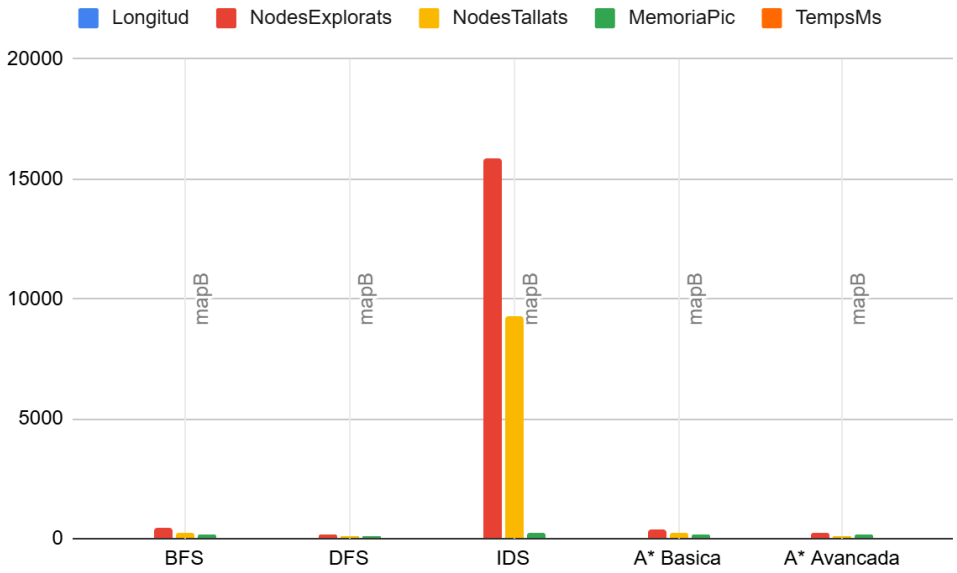
Comparatives de les cerques en cada mapa:

mapA



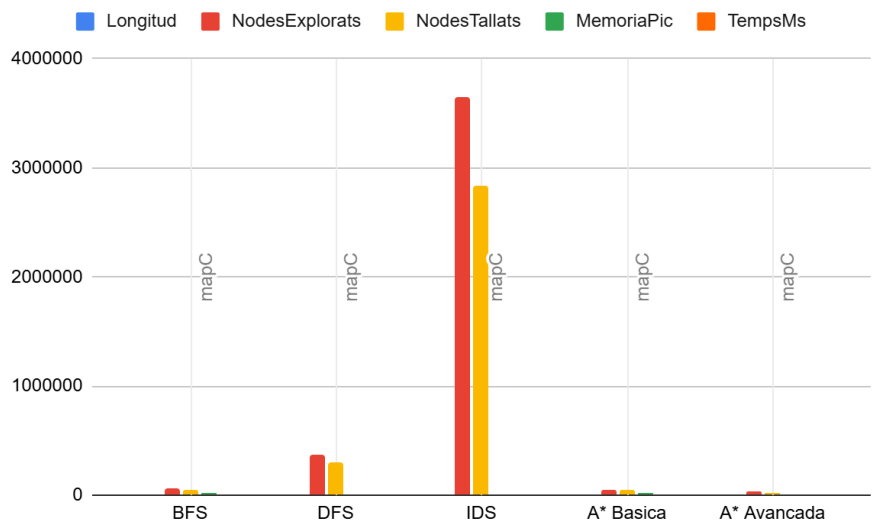
Es veu clarament que A Avançada* és la més eficient en gairebé totes les mètriques, especialment en memòria i temps. DFS és ràpid però explora més nodes. BFS i IDS tenen un comportament intermig.

mapB



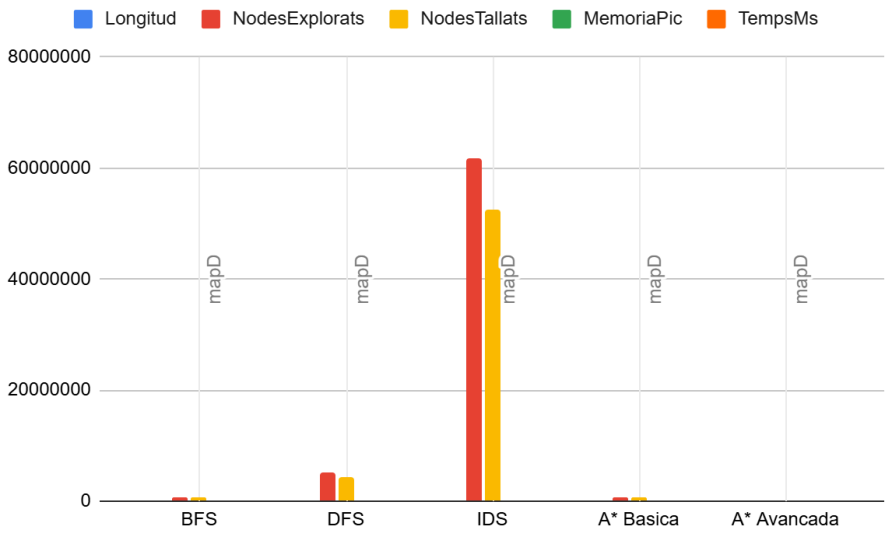
L'algorisme A* Avançada demostra ser el més eficient, amb els valors més baixos en totes les mètriques: nodes explorats, nodes tallats, memòria i temps. DFS destaca per la seva velocitat però a costa d'explorar molts més nodes. BFS i IDS presenten valors intermedis, sent IDS una mica més eficient en memòria que BFS.

mapC



A* Avançada és l'algorisme més eficient, amb valors mínims en totes les categories. A* Bàsica ocupa el segon lloc, notablement millor que BFS, DFS i IDS. DFS és el pitjor algorisme, amb xifres extremadament altes en nodes explorats i memòria, tot i ser ràpid. BFS i IDS mostren resultats intermedis, amb IDS lleugerament més eficient.

mapD



A* Avançada es confirma com l'algorisme òptim, amb diferències significatives respecte als altres. A* Bàsica també mostra excel·lents resultats, molt per sota de la resta. DFS és molt més ineficient en memòria i nodes explorats, tot i el seu bon temps. BFS i IDS presenten valors intermedis molt alts, demostrant la superioritat clara dels algorismes informats (A*) sobre els cegues (BFS, DFS, IDS) en problemes complexos.